FIAP · Engenharia de Software · 4º ano
Global Solution 2026.2 · Desafio CitizenTech

PÁTIO MONITOR

Visão única do pátio para terminais e garagens de ônibus

Secure DevOps Tools & Cloud Computing

Deploy Blindado do Pátio Monitor

Professores: Gabriel Henrique Boyczuk Arcanjo e Cleórbete Santos

Turma: 4ESPV

Integrantes

Camila do Prado Padalino · RM 98316

Felipe Cavalcante Bressane · RM 97688

Gabriel Teixeira Machado · RM 551570

Repositório: github.com/FeBressane/patio-monitor-devops
MVP: patio-monitor-gs-97688.azurewebsites.net

São Paulo, novembro de 2026

1. Ficha Técnica do MVP

Campo	Pátio Monitor
Nome do produto	Pátio Monitor
Cidadão e território	O passageiro que espera na plataforma e o controlador de terminais e garagens de ônibus em São Paulo. O Terminal Pirituba é o cenário de referência.
Problema	Escala, portaria e rastreamento não concordam sobre onde está cada ônibus, e o controlador decide sem saber se o veículo está fora, manobrando ou foi trocado.
Proposta de valor	Uma visão única do pátio que cruza o previsto com o registrado e aponta pendências enquanto ainda dá tempo de agir.
Arquitetura	Front web, API FastAPI, pipeline de dados (Airflow → Databricks), assistente de IA com RAG, detector de visão, robô de conciliação (RPA) e Smart Bus Gate (ESP32) e automação RPA no n8n, publicados por CI/CD no Azure App Service.
Fontes de dados	SPTrans Olho Vivo, GTFS da SPTrans, Open-Meteo e portaria simulada em CSV (regra de geração documentada).
Modelos de IA	Assistente do controlador com LLM e RAG, YOLO para veículos no pátio e o modelo de Machine Learning do Smart Bus Gate.
Indicadores de impacto	Porcentagem de partidas no horário, tempo para localizar um veículo, saídas sem confirmação por dia e tempo médio de permanência.
ODS	ODS 11 (meta 11.2) e ODS 9.
URL do MVP e repositório	https://patio-monitor-gs-97688.azurewebsites.net · Swagger em /docs · repositório github.com/FeBressane/patio-monitor-devops

A mesma ficha abre o README do repositório.

2. Diagrama da arquitetura

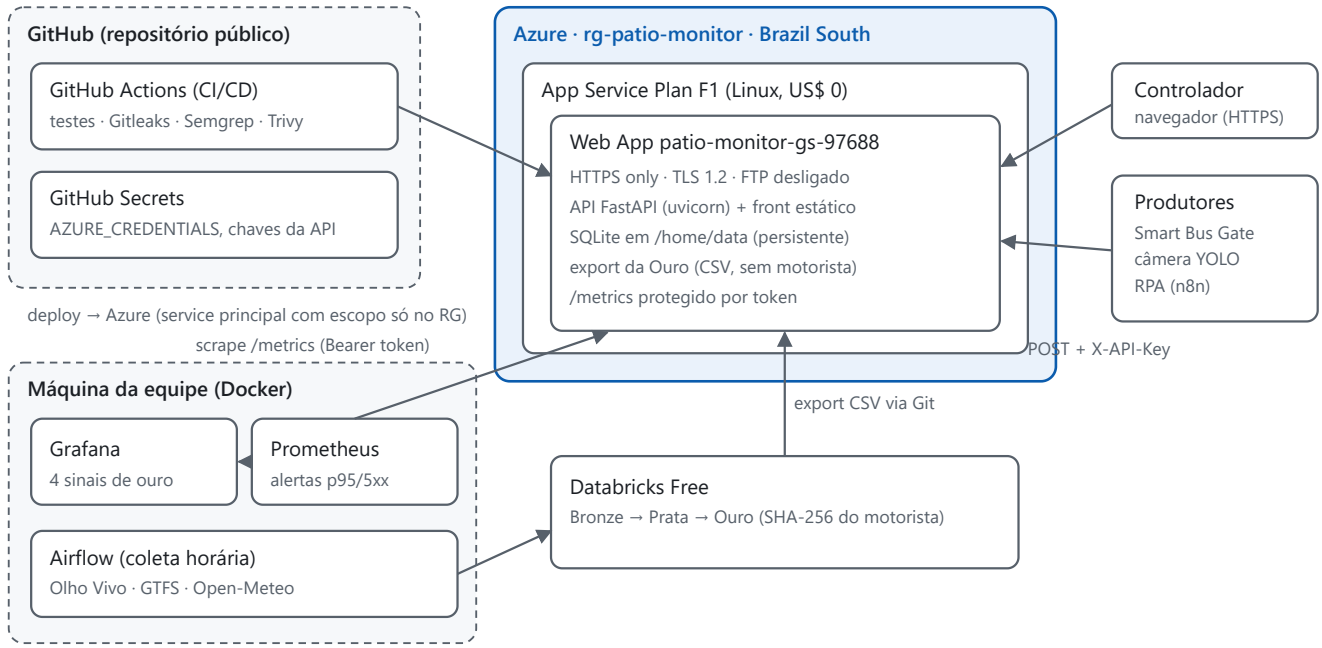


Figura 1. O deploy chega à Azure só pelo pipeline. Usuários e produtores acessam por HTTPS. A observabilidade roda localmente e lê o MVP publicado.

3. Containers (Docker)

- `api/Dockerfile` : **multi-stage**. O 1º estágio instala as dependências num venv, e a imagem final `python:3.12-slim` leva só o venv e o código. Roda como **usuário não root** (`uid 10001`), com `HEALTHCHECK` em `/health` .
- `front/Dockerfile` : front servido por **nginx sem root**, com proxy de `/api` para a API.
- `docker-compose.yml` : sobe **front + API + Postgres**. A API só escuta em `127.0.0.1` , e o acesso público passa pelo front. O profile `monitoramento` sobe Prometheus e Grafana.
- **Nenhum segredo dentro da imagem.** Senhas e chaves vêm de variáveis de ambiente (`.env` local, fora do Git, ou App Settings na nuvem).
- O Trivy varre as duas imagens no pipeline e barra qualquer CVE **HIGH/CRITICAL** com correção disponível. As imagens atuais passam sem nenhuma.

4. Infraestrutura como código (Terraform)

`devops-cloud/terraform/` descreve o ambiente inteiro: resource group, **App Service Plan F1** (Linux) e o **Linux Web App** Python 3.12.

Exigência de segurança	Como o Pátio Monitor atende
Expor só 80/443	O App Service só recebe tráfego HTTP/HTTPS, e `https_only = true` redireciona o HTTP para HTTPS. TLS mínimo 1.2, HTTP/2 ligado.
Porta 22 nunca aberta para 0.0.0.0/0	Não existe SSH público. O console Kudu/SCM só aceita login Entra ID com RBAC, porque a autenticação básica (usuário e senha) de SCM e FTP está **desligada** e o FTP/FTPS está `Disabled` .
Segredos fora do código	O Terraform só grava configurações não secretas. `INGEST_API_KEY` , `METRICS_TOKEN` e `ASSISTENTE_URL` são gravadas pelo pipeline a partir do GitHub Secrets e ficam em `ignore_changes` , para não passarem pelo state.
`.tfvars` fora do repositório	O `.gitignore` bloqueia `*.tfvars` e `*.tfstate` . No repositório só existe o `terraform.tfvars.example` , sem valores reais.
State	O state é local e fica fora do Git. Um backend remoto num Storage Account consumiria crédito da assinatura, e a meta do projeto é custo zero. Para uma equipe maior, o próximo passo é o backend `azurerm` com lock.

> **Por que App Service e não uma VM.** A VM exigiria abrir e manter portas, SO e patches, e custaria crédito (uma B1s fica ligada 24 h até 26/11). O App Service F1 é PaaS: a Azure cuida do SO e do runtime, o custo é US$ 0, e a superfície de ataque é menor, porque não há porta 22 para proteger. A mesma API roda em container no `docker compose` , o que demonstra a parte de containers.

Problemas reais encontrados no provisionamento

- **x509 / `CERTIFICATE_VERIFY_FAILED` :** o "Web/Mail Shield" do antivírus Norton intercepta HTTPS com uma CA própria. O Azure CLI (Python) e o Terraform (Go) não confiam nela. O antivírus intercepta até o TLS

local entre o Terraform e o plugin `azurerm`. A correção foi juntar a raiz do Norton ao `cacert.pem` do Azure CLI (`REQUESTS_CA_BUNDLE`) e rodar o Terraform com `TF_DISABLE_PLUGIN_TLS=1`, que desliga só esse TLS local entre os dois processos (a conexão com a Azure continua em HTTPS).

- **Recursos criados antes do Terraform funcionar:** durante o bloqueio do certificado, o ambiente foi criado com `az` CLI. Depois, os 3 recursos foram trazidos para o código com `terraform import` e o `terraform apply` alinhou a configuração. Desde então o `terraform plan` não mostra diferença: a nuvem é gerida pelo Terraform.
- **Política de regiões do Azure for Students:** só libera brazilsouth, chilecentral, canadacentral, eastus2 e centralus (`RequestDisallowedByAzure` em eastus).
- **Limite de 1 plano F1 Linux por região:** o F1 de Brazil South estava ocupado por um projeto antigo, que foi removido para liberar a região.

[PRINT: `terraform plan` mostrando "No changes" depois do import, ou o `terraform apply` concluído]

5. Pipeline CI/CD DevSecOps (GitHub Actions)

Arquivo: `.github/workflows/ci-cd.yml` . Roda a cada push em qualquer branch e em pull requests. O deploy só acontece na `main` .

Job	O que faz	Barra quando
testes	`pytest` da API (12 testes: health, ingestão com chave, robô movendo ônibus, conciliação...)	algum teste falha
Gitleaks	Varre **todo o histórico** do Git atrás de segredos	encontra token/chave
Semgrep (SAST)	Regras `p/python` , `p/dockerfile` e `p/terraform`	qualquer achado
Trivy (IaC e deps)	Vulnerabilidades, misconfig e segredos no código e no Terraform	HIGH/CRITICAL corrigível
Build + Trivy das imagens	Build da API e do front e scan de cada imagem	HIGH/CRITICAL corrigível
Deploy	Login na Azure com service principal, gravação dos segredos nas App Settings, deploy e **health check** em `/health`	o app não responde em 5 min

- **Segredos:** ficam em GitHub Secrets (`AZURE_CREDENTIALS` , `AZURE_WEBAPP_NAME` , `AZURE_RESOURCE_GROUP` , `INGEST_API_KEY` , `METRICS_TOKEN`) e no environment `producao` .
- **Menor privilégio:** o service principal `sp-patio-monitor-github` é *Contributor* **só no resource group** `rg-patio-monitor` , não na assinatura.
- O deploy só roda depois que testes, Gitleaks, Semgrep e Trivy passam (`needs: build`).

Execução barrada e execução corrigida

No branch `demo/pipeline-barrado` , um token **falso** foi commitado de propósito em `api/app/sptrans_cliente.py` .

Execução	Resultado	Detalhe
#43524648708	Barrada	O Gitleaks encontrou o token, e o build, o scan das imagens e o deploy foram **pulados**.
#43524768351	Corrigida	O token passou a vir de variável de ambiente (`os.getenv`), e o commit foi reescrito para o valor não ficar no histórico. Todos os jobs passaram.

[PRINT: execução #43524648708 com o job Gitleaks em vermelho e o log mostrando o achado]

[PRINT: execução #43524768351 com todos os jobs em verde]

[PRINT: execução da main com o job "Deploy no Azure App Service" e o health check em verde]

6. Observabilidade (Prometheus + Grafana)

A API expõe `/metrics` no formato Prometheus (`prometheus-fastapi-instrumentator`). Na nuvem esse endpoint exige token: sem ele a resposta é **401**. O Prometheus roda localmente pelo `docker compose --profile monitoramento` e lê o MVP publicado por HTTPS com `Bearer` (job `patio-azure` , a cada 30 s, para não gastar a cota de CPU do F1).

Sinal de ouro	Painel no Grafana
Latência	Latência p95 por endpoint (`histogram_quantile` sobre `http_request_duration_seconds`)
Tráfego	Requisições por segundo
Erros	% de respostas 4xx/5xx
Saturação	CPU e memória do processo da API

Alertas (`alertas.yml`):

- `LatenciaP95PosicoesAlta` : p95 de `/posicoes` acima de **300 ms** por 2 min. É assim que a promessa de baixa latência do produto é medida, em vez de ser só afirmada.
- `TaxaDeErros5xx` : mais de 5% de respostas 5xx.
- `ApiFora` : alvo sem responder por 1 min.

O que foi medido

- **Latência:** com cerca de 800 requisições reais em 5 minutos (13/11/2026, 21h), o p95 de `/api/posicoes` e dos demais endpoints ficou em **~95 ms**, abaixo da meta de 300 ms do alerta.
- **Alerta disparado (teste controlado de queda):** às 21:24 o App Service foi parado de propósito (`az webapp stop`). O `up` do job `patio-azure` foi a 0, o alerta `ApiFora` passou de *pending* para ***firing***, e às 21:30 o app foi religado (`/health` voltou com 200). O painel "Alertas do Prometheus" do dashboard mostra o disparo.

[PRINT: dashboard "Pátio Monitor · 4 sinais de ouro" com dados do MVP publicado]

[PRINT: painel "Alertas do Prometheus" (ou Prometheus › Graph com a consulta `ALERTS` , última 1 h) mostrando ApiFora em firing por volta das 21:28 de 13/11]

7. Modelo de responsabilidade compartilhada

O App Service é **PaaS**: a Azure assume a infraestrutura e o sistema operacional, e a equipe responde por tudo que coloca e configura em cima.

Camada	Azure (provedor)	Equipe Pátio Monitor
Datacenter, rede física, hardware	Total	—
SO do host, hipervisor e patches do runtime Python	Total	Escolher uma versão suportada (3.12)
Rede e TLS do app	Certificado `*.azurewebsites.net` e balanceador	HTTPS only, TLS 1.2, FTP desligado
Identidade e acesso	Entra ID e RBAC (o serviço)	Service principal com escopo só no RG, sem senha básica no SCM/FTP
Código e dependências	—	Total: testes, Semgrep, Trivy e Gitleaks no pipeline
Segredos	Guarda as App Settings cifradas	Origem no GitHub Secrets, rotação e nada no Git
Dados (SQLite, CSV da Ouro)	Disco persistente do `/home`	Classificação, minimização e LGPD
Monitoramento	Métricas da plataforma	Métricas da aplicação, painel e alertas

8. Dados pessoais (LGPD): onde ficam

O único dado pessoal do domínio é o do **motorista** (matrícula e nome). No projeto ele é **fictício**: é gerado pela portaria simulada com nomes do tipo "Motorista A".

Lugar	Dado do motorista
Portaria simulada (pasta `chegada/` do repositório patio-monitor-bigdata, fora do Git) e camadas Bronze/Prata no Databricks	Texto (fictício)
Camada Ouro (`fato_permanencia`) no Databricks	**Só hash SHA-256** da matrícula e do nome
Export CSV da Ouro, API, SQLite do App Service, Postgres do compose	**Nenhum**: o export não tem as colunas, e a API ainda filtra qualquer coluna com "motorista"
Logs da API e do assistente	Nenhum

Limite conhecido: SHA-256 sem sal de um nome pode ser revertido por dicionário. Com dados reais, o certo seria HMAC-SHA-256 com a chave num cofre (Azure Key Vault), e não um hash puro.

9. Estimativa de custo mensal

Preços de tabela da Azure Retail Prices API para Brazil South, em USD, consultados em 13/11/2026. Mês = 730 h.

Recurso	Plano	US$/hora	US$/mês
App Service Plan (Linux), **usado no MVP**	F1 Free	0,00	**0,00**
Web App, SQLite em `/home`, certificado HTTPS	incluso no plano	—	0,00
GitHub Actions (repositório público)	gratuito	—	0,00
Prometheus + Grafana (locais)	máquina da equipe	—	0,00
Total do MVP, ligado até a correção (~26/11)			**US$ 0,00**
Cenário piloto num terminal (sem cota de CPU, com Always On)	B1 Linux	0,02	14,60
Cenário com escala automática	P0v3 Linux	0,111	81,03

Limites do F1, assumidos conscientemente: 60 min de CPU por dia, 1 GB de RAM e sem Always On. O app dorme depois de cerca de 20 min sem acesso, e a primeira requisição demora alguns segundos. Para o pitch e a correção isso basta. Num piloto real, o B1 resolve por cerca de US$ 15/mês.

10. Conclusão

O Pátio Monitor está publicado numa URL pública em HTTPS, com custo zero, e só chega à nuvem por um pipeline que testa, procura segredos, faz análise estática e varre dependências e imagens antes do deploy. A execução barrada mostra o pipeline impedindo que um segredo vá para produção. O Grafana mede a latência que o produto promete, em vez de só afirmá-la.

Links: repositório github.com/FeBressane/patio-monitor-devops · MVP patio-monitor-gs-97688.azurewebsites.net · Swagger /docs · pipeline Actions